

Instructions

- Homework 4 is due Monday, August 17th at 18:00 Chicago Time.
 - We will not accept any submissions past 18:00:00, even if they are only one second late.
 - You **must** upload the following files to the class Canvas:
 - LASTNAME_FIRSTNAME.pdf
 - LASTNAME_FIRSTNAME.ipynb
 - Your code notebook **must** be runnable using my environment outlined in class 1 (Python 3.14, and the `requirements.txt`).
 - You **must** use this template file and fill out your solutions for the written portion.
 - Please note that your last name and first name should match what you appear on Canvas as.
 - Include code snippets where required, as well as math and equations.
 - Be *concise* where possible, all of the homework problems can be answered in a few lines of math, code, and words.
 - Each problem gives you the data code and the solver settings. Use them as written. Then your numbers will agree with your classmates' numbers.
-

Problem 1: New Data in Our Curve

We'll start by fitting a curve made up of pieces (piecewise linear). Where these pieces intersect are our knots. With K knots the model is:

$$f(x) = w_0 + w_1x + \sum_{j=1}^K w_{1+j}(x - \kappa_j)_+, \quad (u)_+ = \max(u, 0)$$

Each column $(x - \kappa_j)_+$ is zero before the knot κ_j . After the knot it has a slope of 1, so that w_{1+j} is the change in slope at that knot. The model has $p = 2 + K$ parameters.

```

1 import numpy as np
2
3 def basis(x, knots):
4     """Build the basis matrix. One extra column per knot."""
5     cols = [np.ones_like(x), x] + [np.clip(x - k, 0.0, None) for k in knots]
6     return np.column_stack(cols)
7
8 def knots(K):
9     """K knots, evenly spaced inside (0, 1)."""
10    return np.linspace(0, 1, K + 2)[1:-1]
11
12 def fit_ols(A, y):
13     beta, *_ = np.linalg.lstsq(A, y, rcond=None)
14     return beta

```

The x values are the same for all draws; note that only the noise is random (same as Homework 2).

```

1 N_TRAIN, N_TEST = 60, 500
2 SIGMA = 0.3
3 K_MAX = 18
4 N_REPS = 600
5
6 x_train = np.linspace(0, 1, N_TRAIN)
7 x_test = np.linspace(0.005, 0.995, N_TEST)
8
9 def f_true(x):
10    return np.sin(2 * np.pi * x)
11
12 # in repetition s, use rng = np.random.RandomState(s) and draw
13 # y_train = f_true(x_train) + rng.normal(0, SIGMA, N_TRAIN)
14 # y_test = f_true(x_test) + rng.normal(0, SIGMA, N_TEST)

```

Problem 1.1: Basis Functions

- Plot the columns of `basis(x_train, knots(4))` against x .
- In one sentence, what does the coefficient on the column $(x - \kappa_j)_+$ tell you?

Answer:

Problem 1.2: One Set of Data

Use `np.random.RandomState(0)` for the noise. Fit the model for $K = 1$, $K = 4$ and $K = 18$.

- Plot the training data and the three curves on one plot.
- Just from eyeballing; which do you think is the best from balancing bias and variance?

Answer:

Problem 1.3: Sim

For each $K = 0, 1, \dots, 30$ and each repetition $s = 0, \dots, 999$:

1. Draw new `y_train` and `y_test` with `np.random.RandomState(s)`.
2. Fit the model on the training data only.
3. Record the MSE on the training data and the MSE on the test data.

Take the mean of each over the 600 repetitions.

- Plot the two mean curves against K on one plot (ie. mean MSE and test MSE for each K).
- Give a table of the two values for $K \in \{0, 2, 3, 5, 10, 15, 30\}$.

Answer:

Problem 1.4: What the Plot Shows

- Does the training MSE ever increase when you add a knot?
- In one sentence, why is that?
- Give the value of K with the lowest test MSE.

Answer:

Problem 1.5: Theory

For a model with p parameters that has the correct shape, the expected test MSE is roughly:

$$\sigma^2 \left(1 + \frac{p}{n}\right)$$

Here $p = 2 + K$ and $n = 60$.

- Draw this theoretical curve on top of your test MSE curve (from the previous question).
- Give the range of K where $|\text{simulated/formula} - 1| \leq 0.03$.
- In 1–2 sentences, what does the formula leave out at small K ?

Answer:

Problem 2: Which Knots Are Real?

In the previous question, the data as a sin wave. This time, the data really is piecewise linear; the slope changes at $x = 0.3$ and at $x = 0.7$. There are 19 possible knots, and we need to see if we can find the real ones.

```

1 TRUE_KNOTS = [0.3, 0.7]
2 TRUE_BETA = np.array([0.0, 2.0, -8.0, 10.0]) # intercept, slope, then the two
      slope changes
3 CANDIDATE_KNOTS = np.linspace(0.05, 0.95, 19)
4 N2, SIGMA2 = 200, 0.3
5
6 rng = np.random.RandomState(7)
7 x2 = rng.uniform(0, 1, N2)
8 y2 = basis(x2, TRUE_KNOTS) @ TRUE_BETA + rng.normal(0, SIGMA2, N2)
9
10 # scikit-learn adds its own intercept, so remove the first column
11 A_full = basis(x2, CANDIDATE_KNOTS)
12 design = A_full[:, 1:]

```

Problem 2.1: Plain OLS

Fit OLS with all 19 possible knots.

- Give the largest coefficient of the 19 knot columns, in absolute value.
- Plot the fitted curve.
- Can you find the two real knots from the coefficients? Give a why or why not reason in one sentence.

Answer:

Problem 2.2: Lasso to the Rescue?

Fit `LassoCV(cv=5, random_state=0, max_iter=200000)` on `design`.

- Give the α/λ that it selects.
- Give the number of knots with a coefficient that is not zero (or extremely close to it). Are both real knots in the list?

Answer:

Problem 2.3: Change the Penalty

Fit `Lasso(alpha=a, max_iter=200000)` for $a \in \{0.001, 0.005, 0.01, 0.02, 0.05, 0.1\}$.

- Give a table with one row for each α : the number of knots that stay, and their values.

- Is there an α that keeps exactly $\{0.3, 0.7\}$ and nothing else?

Answer:

Problem 2.4: Why the Two Disagree

In a few sentences: why do we tend to keep more knots than are really needed?

Answer:

Problem 3: Your Own Basis, and a Rule That OLS Cannot Follow

Like we saw in class, Gaussian blobs evenly spaced by years can be hard to fit a yield curve to. The reason is that most of the "interesting" part of the yield curve happens in the first few years to maturity.

A lognormal basis function is a Gaussian blob in log tenor:

$$\phi_i(\tau) = \exp\left(-\frac{(\log \tau - \mu_i)^2}{2s^2}\right)$$

```

1 def lognormal_basis(tau, mu, s):
2     return np.exp(-((np.log(tau) - mu) ** 2) / (2 * s ** 2))
3
4 TENORS = np.array([0.25, 0.5, 0.75, 1, 2, 3, 4, 5, 7, 10, 12, 15, 20, 25, 30],
5                    dtype=float)
6 CENTRES = np.log(np.array([0.5, 1.5, 4.0, 12.0, 30.0]))
7 WIDTH = 0.9
8
9 def true_curve(tau):
10     return 0.02 + 0.01 * np.sqrt(tau)
11
12 rng = np.random.RandomState(0)
13 y3 = true_curve(TENORS) + rng.normal(0, 0.0004, len(TENORS))
14
15 tau_fine = np.linspace(0.25, 30, 500)
16
17 def ln_design(tau):
18     return np.column_stack([lognormal_basis(tau, m, WIDTH) for m in CENTRES])
19
20 A3 = ln_design(TENORS)

```

Problem 3.1: The Basis

- Plot the five basis functions against τ on `tau_fine`.
- Take the blob at 0.5 years and the blob at 30 years. For each one, give (roughly) the width in years over which the blob is at or above 0.5.

Answer:

Problem 3.2: Ordinary Least Squares

Fit the five weights with OLS on `A3` and `y3`. Store them as `w_ols3`.

- Give the five weights, and the number of them that are negative.
- In one sentence, why is a negative weight hard to explain when each basis function is a blob of yield?

Answer:

Problem 3.3: Non-Negative Weights

In class we said that we could stop the negative weights with the rule $w \geq 0$. Apply that rule:

```
1 from scipy.optimize import minimize, nnls
2
3 res = minimize(lambda w: np.sum((A3 @ w - y3) ** 2), np.maximum(w_ols3, 0.0),
4                 method="L-BFGS-B", bounds=[(0.0, None)] * A3.shape[1],
5                 options={"maxiter": 100000, "ftol": 1e-18, "gtol": 1e-14})
```

- Give the five new weights.
- Give the RMSE against `true_curve(tau_fine)`.
- Plot your two fitted yield curves against the data, which do you think looks better?

Answer:

Problem 4: Switching Up Losses

Sometimes, data might have bad values.

```

1 x4 = np.linspace(0, 1, 120)
2 clean_curve = np.sin(2 * np.pi * x4)
3
4 rng = np.random.RandomState(5)
5 y4 = clean_curve + rng.normal(0, 0.05, 120)
6 bad_idx = rng.choice(120, 8, replace=False)
7 y4[bad_idx] += rng.choice([-1, 1], 8) * 1.5      # 8 bad values
8
9 B4 = basis(x4, knots(8))                        # 10 parameters
10 w_ols = fit_ols(B4, y4)                        # the start point for both fits below

```

Use these settings for every fit in this problem. Start each fit from the OLS answer.

```

1 from scipy.optimize import minimize
2
3 OPT = {"maxiter": 50000, "maxfun": 200000, "ftol": 1e-15, "gtol": 1e-12}
4 # minimize(loss, w_ols, method="L-BFGS-B", options=OPT)

```

Problem 4.1: Least Squares

Plot the data, the OLS curve, and `clean_curve` on one set of axes. Show the 8 bad values in a different color.

Answer:

Problem 4.2: Two Other Losses

Write these two losses yourself, then run them through `scipy.optimize.minimize`:

- the Huber loss with $\delta = 0.1$:

$$\rho_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta (|r| - \frac{1}{2}\delta) & |r| > \delta \end{cases}$$

- the (almost) absolute loss. Use the smooth version below (so that it's differentiable at $r = 0$).

$$\rho(r) = \sqrt{r^2 + \varepsilon^2}, \quad \varepsilon = 10^{-3}$$

Plot these two curves and the OLS curve from 4.1 together with `clean_curve`.

Answer:

Problem 4.3: Which Loss Wins

- Give the RMSE of the three fits against `clean_curve`.
- Which one is lowest, and why?

- Now give the RMSE of the three fits against `y4` as well. Which is better, and why do you think that is?

Answer:

Problem 4.4: The Effect of δ

Fit the Huber loss again for $\delta \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0\}$.

- Give the RMSE against `clean_curve` for each δ .
- What is Huber loss like for very small δ ? And what is it like for really big δ ?

Answer: